Зависимые имена шаблонов

– имена, зависящие каким-либо образом от параметров шаблона

```
struct Test {
    struct A {
        int a = 0;
        int b = 0;
    };
};

void f() {
    Test::A* x;
}

int main () {
    f();
}
```

```
template <typename T>
struct Test {
    struct A {
        T a = 0;
        T b = 0;
    };
};

template <typename T>
void f() {
    Test<T>::A* x;
}

int main () {
    f<int> (); //!!!
}
```

В данном контексте ”A* x” может восприниматься и как выражение, и как объявление. По умолчанию будет восприниматься как выражение, а A – статической переменной.

```cpp
template <typename T>
struct Test {
    struct A {
        T a = 0;
        T b = 0;
    };
};

template <>
struct Test<int> {
    static int A;
};

int Test<int>::A = 5;
int x = 9;

template <typename T>
void f() {
    Test<T>::A* x; //ok
}
int main () {
    f<int> ();
}
```

```cpp
template <typename T>
struct Test {
    struct A {
        T a = 0;
        T b = 0;
    };
};

template <>
struct Test<int> {
    static int A;
};

int Test<int>::A = 5;
int x = 9;

template <typename T>
void f() {
    Test<T>::A* x; //error
}
int main () {
    f<double> ();
}
```

Компилятор не может в зависимости от параметра шаблона определить, это выражение или определение

```
template <typename T>
struct Test {
    struct A {
        T a = 0;
        T b = 0;
    };
};

template <typename T>
void f() {
    typename Test<T>::A* x;
    x = new typename Test<T>::A;
    std::cout << x->a << std::endl;
}
int main () {
    f<double> ();
    f<int>();
}
```

Если происходит объявление параметра, перед зависимым именем следует писать typename

А если зависимая шаблонная функция?

```
template<typename T>
struct X {
    void setA() {std::cout <<"X";}
};

template<typename T>
void f () {
    X<T> x;
    x.setA();
}

int main () {
    f<double> ();
    f<int>();
}
```

```
template<typename T>
struct X {
    template<typename U>
    void setA() {std::cout <<"X";}
};

template<typename T>
void f () {
    X<T> x;
    x.setA<T>(); //error
}
int main () {
    f<double>();
    f<int>();
}
```

Компилятор видит x.setA (вызов функции), а потом <T>. Для него < — это оператор сравнения "меньше", а не начало списка шаблонных параметров. Он пытается разобрать как (x.setA < T) > ()

Устранение неоднозначности

```
emplate<typename T>
struct X {
    template<typename U>
    void setA() {std::cout <<"X";}
};

template<typename T>
void f () {
    X<T> x;
    x.template setA<T>();
}

int main () {
    f<double> ();
    f<int>();
}
```

Зависимые имена при наследовании

```
template<typename T>
struct Base{
    int x = 0;
};

template<typename T>
struct Derrived: Base<T> {
    void f() {
        ++x;
    }
};

int main () {
    Derrived<int> d;
}
```

error: use of undeclared identifier 'x'

Компиляция шаблонов происходит, условно, в два этапа

Этап 1: Парсинг (без инстанцирования)

Компилятор читает шаблон Derived<T> и проверяет синтаксис и независимые имена (те, которые не зависят от T).

Этап2: Инстанцирование

Только на этом этапе компилятор подставляет T = int

Решение. Нужно явно указать, что x — это зависимое имя

```
template<typename T>
struct Base{
    int x = 0;
};

template<typename T>
struct Derrived: Base<T> {
    void f() {
        ++Base<T>::x;
    }
};

int main () {
    Derrived<int> d;
}
```

```
template<typename T>
struct Base{
    int x = 0;
};

template<typename T>
struct Derrived: Base<T> {
    void f() {
        ++this->x;
    }
};

int main () {
     Derrived<int> d;
}
```

auto

auto позволяет компилятору самостоятельно определить тип переменной на основе ее инициализации. Использование auto делает код более кратким и читаемым, особенно в случаях, когда тип данных длинный или сложный.

Принцип выведения типа схож с процессом вывода шаблонного параметра в шаблонных функциях.

```
auto fcomp (char x, char y) {
    return x == y;
}

int main() {
    auto c1 = 'a’;
    auto& c2 = c1;
    std::cout << fcomp(c1,c2);
}
```

auto

```
template <auto N> //можно указывать auto у шаблонных параметров
struct myTest{
    size_t a = N;
};

//для удобочитаемости или для явного определения можно явно указать возвращаемый тип
auto g(std::vector<myTest<20>>& v) -> std::vector<int> {
    std::vector<int> result;
    for (const auto& item : v) {
        result.push_back(item.a * 2);
    }
    return result;
}
auto fcomp (char x, char y) {//можно указывать auto у возвращаемого типа функции
    return x == y;
}
int main() {
    auto c1 = 'a'; //можно указывать auto у переменной
    auto& c2 = c1; //auto «отбрасывает» ссылки
    std::cout << fcomp(c1,c2);
    std::vector<myTest<20>> m1; // std::vector<auto> m1 не получится сделать
    auto m2 = m1;
    g(m2);
}
```

decltype

decltype позволяет определить тип, исходя из переданной переменной или выражения. delctype формирует тот тип, который имеет выражение в круглых скобках. При этом выражение в круглых скобках не является выполняемым.

```
template <typename Container>
auto getElement (Container& cont, size_t index)
         -> decltype(cont[index]) {
    return cont[index];
}

int main() {
    int x = 0;
    decltype(x) p1 = 0;
    decltype(throw 1)* p2 = &x; //expression with side effects
                                // has no effect in an unevaluated context
    decltype((x)) p = 0; //p – имеет тип int&
    decltype(++x) y = x; //y – имеет тип int&
    ++y;
    std::cout << x;
}
```

Ещё пример

```
int main() {
    int x = 0;
    decltype(x) p1 = 0;

    decltype((x)) p = 0;    //p – имеет тип int&
    decltype(x + p1) y = x; //y – имеет тип int
    ++y;
    std::cout << x;
}
```

decltype vs auto

decltype не отбрасывает ссылки, его можно строить от выражения, он называет точный тип.

```
int main() {
    int x = 0;
    int& y = x;
    decltype(y) z = y;
    decltype(x)& k = x;

    ++x;
    std::cout << x << ' ' << y << ' ' << z << ' ' << k;
}
```

```
int main() {
    const int x = 13;

    auto a = x;         // int (const отброшен)
    decltype(x) d = x; // const int (const сохранён)

    // d = 100; нельзя менять константу
    a = 100;
    std::cout << x << ' ' << a << '\n';

    int arr[5] = {1, 2, 3, 4, 5};

    auto a1 = arr;       // int*
    decltype(arr) d1;    // int[5] (массив сохраняется, но содержимое, конечно, нет)
    a1[2] = 1;  // OK
    d1[2] = 2;  // OK
    a1 = d1;
    // d1 = a1; нельзя
    std::cout << a1[2] << ' ' << d1[2] << '\n';
    int x1 = 13;

    decltype(x1) d2;    // int
    decltype((x1)) d3 = x1; // int&

    auto a3 = x1;       // int
    auto a4 = (x1);     // int (скобки не влияют)
}
```

Метапрограммирование

Под метапрограммированием в языке C++ подразумеваются вычисления на стадии компиляции. Основным инструментом для этого являются шаблоны классов, называемые в этом случае (в случае их использования для вычислений на стадии компиляции) метафункциями.

Параметрами таких метафункций служат параметры шаблонов классов, а значения метафункций (их может быть несколько) хранятся в статических (static) переменных или определениях типов (typedef) класса.

Метапрограммирование

Шаблонное метапрограммирование обладает полнотой по Тьюрингу, поскольку оно поддерживает выбор (if-eLse) и циклы (посредством рекурсии). Следовательно, теоретически в метапрограммах можно выполнять любые вычисления.

Шаблонное метапрограммирование является чисто функциональным языком, в нём отсутствует оператор присваивания значений переменным, переменные возможны только в качестве параметров метафункций. Как и в других чисто функциональных языках программирования, единственно возможный способ организации циклов – это рекурсия.

Простой пример. Факториал

```
template <unsigned long N>
struct factorial {
    static const unsigned long value = N * factorial<N-1>::value;
};

template <>
struct factorial<0> {
    static const unsigned long value = 1;
};

int main () {
    unsigned long f5 = factorial<10>::value;
    std::cout << f5;
}
```

Константа на момент компиляции

```
template <size_t N, size_t M>
class Matrix { };

template <unsigned long N>
struct factorial {
    static const unsigned long value = N * factorial<N-1>::value;
};

template <>
struct factorial<0> {
    static const unsigned long value = 1;
};

int main () {
    int y = 0;
    std::cin >> y;
    const int x = y;
    int a[x]; //ok
    unsigned long f5 = factorial<x>::value; //error
    Matrix<x,x> m;  //error
}
```

constexpr
константа, известная на этапе компиляции

```
//см. выше
int f(int r = 0){
    return r*5;
}
int main () {
    const int number = 10;
    int array[number*3-6];
    int a[f(3)]; //ok
    unsigned long f5 = factorial<f(1)>::value; //error
    Matrix<f(2),f(1)> m; //error
}
```

```
constexpr int f(int r = 0){
    return r*5;
}
int main () {
    const int number = 10;
    int array[number*3-6];
    int a[f(3)]; //ok
    unsigned long f5 = factorial<f(1)>::value; //ok
    Matrix<f(2),f(1)> m; //ok
}
```

Поэтому лучше так

```
template <unsigned long N>
struct factorial {
    static constexpr unsigned long value = N * factorial<N-1>::value;
};

template <>
struct factorial<0> {
    static constexpr unsigned long value = 1;
};

int main () {
    constexpr int number = 10;
    unsigned long f5 = factorial<number>::value;
    std::cout << f5;
}
```

Метафункции. Пример 1

```
template <typename T, typename U>
struct is_same {
	static constexpr bool value = false;
};

template <typename T>
struct is_same <T,T> {
	static constexpr bool value = true;
};
```

Позволяет узнать на этапе компиляции, одинаковые или различные типы используются

Если вспомнить про пример с зависимыми именами шаблона

```
template <typename T>
struct Test {
    struct A {
        T a = 0;
        T b = 0;
    };
};

template <>
struct Test<int> {
    static int A;
};

int Test<int>::A = 5;
int x = 9;
```

```
template <typename T>
void f() {
    if (is_same<int,T>::value) {
        std::cout << Test<T>::A* x;
    }
    else {
        typename Test<T>::A* x;
        x = new typename Test<T>::A;
        std::cout << x->a;
    }
}
int main () {
    f<int>();
    f<double>();
}
```

Это всё равно пока работать не будет, потому что компилятор проверяет функцию на общие ошибки, то есть это условие if будет проверено по общим правилам, в итоге получится та же ошибка, что и на 4 слайде

if constexpr (expr). Компилировать только ту ветку, где expr == true

А вот теперь будет. Потому что к if добавилось ключевое слово constexpr, таким образом, это условие будет просчитано на этапе компиляции.

```
template <typename T>
struct Test {
    struct A {
        T a = 0;
        T b = 0;
    };
};

template <>
struct Test<int> {
    static int A;
};

int Test<int>::A = 5;
int x = 9;
```

```
template <typename T>
void f() {
    if constexpr (is_same<int,T>::value) {
        std::cout << Test<T>::A* x;
    }
    else {
        typename Test<T>::A* x;
        x = new typename Test<T>::A;
        std::cout << x->a;
    }
}
int main () {
    f<int>();
    f<double>();
}
```

Метафункции. Пример 2

```
template <typename T>
struct remove_reference {
    using type = T;
};

template <typename T>
struct remove_reference<T&> {
    using type = T;
};

template <typename T>
void f(T t) {
    T test; //error
    typename remove_reference<T>::type test1; //ok
}
int main () {
    double x = 0.9;
    f<double&>(x);
}
```

Метафункции. Много примеров

cppreference.com Create account Search

Page Discussion Standard revision: Diff View Edit History

C++ Standard library headers

Standard library header <type_traits> (C++11)

This header is part of the metaprogramming library.

Classes

Helper Classes

integral_constant (C++11) bool_constant (C++17)	compile-time constant of specified type with specified value (class template)
true_type	std::integral_constant<bool, true>
false_type	std::integral_constant<bool, false>

Primary type categories

is_void (C++11)	checks if a type is void (class template)
is_null_pointer (C++11)(DR*)	checks if a type is std::nullptr_t (class template)
is_integral (C++11)	checks if a type is an integral type (class template)
is_floating_point (C++11)	checks if a type is a floating-point type (class template)
is_array (C++11)	checks if a type is an array type (class template)
is_enum (C++11)	checks if a type is an enumeration type (class template)
is_union (C++11)	checks if a type is a union type (class template)
is_class (C++11)	checks if a type is a no (class template)
is_function (C++11)	checks if (cl

Начиная с C++17, чтобы не писать длинные выражения типа
is_same<int,T>::value
, можно использовать готовые метафункции:
std::is_same_v

А вместо
typename remove_reference<T>::type
писать
std::remove_reference_t

и т.д.

SFINAE. Substitution Failure Is Not An Error

Если в момент инстанцирования объявления функции компилятор получает некорректный тип, то это не ошибка компиляции, а просто нерассматриваемая версия, и выбирается следующая, более подходящая.

Рассмотрим пример:

```
class MyArray {
    int data[3] = {10, 20, 30};
public:
    using iterator = int*; // Вложенный тип iterator (указатель на int)
    iterator begin() { return data; }
    iterator end() { return data + 3; }
};

template <typename T>
void print(T container) {
    std::cout << "Values:{ ";
    for(auto i = container.begin(); i< container.end(); i++)
        std::cout << value << " ";
    std::cout << "}\n";
}
int main (){
    MyArray arr;
    print(arr);
    int i = 7;
    print(i);
}
```

error: invalid range expression of type 'int'; no viable 'begin' function available

Поэтому логичней сделать так

```
template <typename T, typename = typename T::iterator>
void print(T container) {
    std::cout << "Values:{ ";
    for(auto i = container.begin(); i< container.end(); i++)
        std::cout << *i << " ";
    std::cout << "}\n";
}
int main (){
    MyArray arr;
    print(arr);
    int i = 7;
    print(i); //Error
}
```

1. Выводится тип параметра T, он будет int
2. Происходит попытка вывода второго параметра, который не имеет имени: int::iterator. Но компилятор не может найти тип iterator в int и поэтому попытка вывода проваливается
3. Функция исключается из рассмотрения
4. Т.к. больше никаких функций у нас нет, компиляция завершается ошибкой.

no matching function for call to 'print'

?

```
template <typename T>
void print(T container) {
    std::cout << "Values:{ ";
    for(auto i = container.begin(); i< container.end(); i++ )
        std::cout << *i << " ";
    std::cout << "}\n";
}

template <typename T>
void print(T i) {
    std::cout << i << " ";
}
```

call to 'print' is ambiguous

А если так?

```
template <typename T, typename = typename T::iterator>
void print(T container) {
    std::cout << "Values:{ ";
    for(auto i = container.begin(); i< container.end(); i++ )
        std::cout << *i << " ";
    std::cout << "}\n";
}

template <typename T>
void print(T i) {
    std::cout << i << " ";
}

int main (){
    MyArray vec;
    print(vec); //!!! call to 'print' is ambiguous

    int i = 7;
    print(i); //ок
}
```

std::enable_if

```
template <bool condition, typename Type>
struct enable_if {};

template <typename Type>
struct enable_if<true, Type>{
    using type = Type;
};
```

Если первым аргументом шаблона будет false, тогда у результирующего класса enable_If не будет члена type, а если будет true, — тогда будет

Поэтому, в принципе, неважно, какой тип определит enable_if. Данная метафункция в зависимости от условия либо выводит тип, либо нет. Если вывести тип у компилятора не выходит, ошибки компиляции не возникает, шаблонная функция просто вычёркивается из списка рассматриваемых.

```
template <bool B, typename Type>
struct enable_if {};

template <typename Type>
struct enable_if<true, Type>{
    using type = Type;
};
template <bool B, typename T>
using enable_if_t = typename enable_if<B,T>::type;

template <typename T, enable_if_t<!std::is_integral_v<T>,bool> = true>
void print(T container) {
    std::cout << "Values:{ ";
    for(auto i = container.begin(); i< container.end(); i++ )
        std::cout << *i << " ";
    std::cout << "}\n";
}

template <typename T, enable_if_t<std::is_integral_v<T>,bool> = false>
void print(T i) {
    std::cout << i << " ";
}

int main (){
    MyArray vec;
    print(vec);
    int i = 7;
    print(i);
}
```

В указанном выше слайде используется ещё одна метафункция is_integral, которая возвращает true, если передаваемый тип T является простым скалярным типом. А если T – другой какой-то тип, возвращает false. Таким образом, если функция print принимает вектор, is_integral вернёт false, результат !is_integral будет true, то есть шаблонная метафункция enable_if будет специализирована enable_if<true, Type>, то есть теперь определено type = Type. Выходит, что в таком случае для первой функции print будут выведены шаблонные параметры print<std::vector, bool = true>, что является корректным. Обратите внимание, второй шаблонный параметр (bool) нигде не используется, он существует только для проверки возможности вывода типа, поэтому по умолчанию может принимать любое значение, важен только факт, что тип выведен. При попытке определить шаблонные параметры у второй функции, компилятор не сможет вывести второй шаблонный параметр, и вторая функция в случае работы с вектором даже не будет рассматриваться.

Аналогично будет происходить выбор функции в случае с int. Метафункция is_integral вернёт true, !is_integral будет false, в структуре enable_if<false, Type> ничего нет, type не определён, то есть шаблонные параметры у первой функции определить не удалось, функция исключается из рассмотрения. Во второй функции шаблонные параметры выведены, так как enable_if<true, Type>, выходит, print<int, bool = false>, функция будет вызвана.

Вместо bool можно выбрать любой простой тип, а вместо значений по умолчанию – любое значение. Важен лишь факт вывода типа, а ему просто присваивается какое-то значение

Concepts (с C++20)

Концепты позволяют установить ограничения для параметров шаблонов (как шаблонов функций, так и шаблонов класса)

```
template <параметры>
concept имя_концепта = ограничения;
```

Ограничения представляют условные выражения, которые возвращают значение типа bool – если параметр типа удовлетворяет условию, то возвращается true.

```
template<typename T>
concept Integral = std::is_integral_v<T>;
```

Чтобы определить, соответствует ли определенный тип концепту, надо использовать выражение концепта, которое состоит из имени концепта, после которого в угловых скобках указываются аргументы для параметров шаблона:

```
template<typename T> requires (Integral<T>)
void print(T i) {
    std::cout << i << " ";
}
```

```
class MyArray {
    int data[3] = {10, 20, 30};
public:
    using iterator = int*;
    iterator begin() { return data; }
    iterator end() { return data + 3; }
};

template<typename T>
concept Integral = std::is_integral_v<T>;

template<Integral T>  //можно сократить
void print(T i) {
    std::cout << i << " ";
}

template <typename T> requires (!Integral<T>)
void print(T container) {
    std::cout << "Values:{ ";
    for(auto i = container.begin(); i< container.end(); i++ )
        std::cout << *i << " ";
    std::cout << "}\n";
}
int main (){
    MyArray vec;
    print(vec);
    int i = 7;
    print(i);
}
```

Встроенные концепты (#include <concepts>)

```
#include <iostream>
#include <concepts>

template<std::integral T>
T process(T x) { return x * 2; }


template<std::floating_point T>
T process(T x) { return x / 2; }


template <typename T>
concept sum_types = std::same_as<T, int> || std::same_as<T, double>;

template <sum_types T>
T sum(T a, T b){ return a + b;}

int main() {
    std::cout << process(10) << std::endl;   // 20
    std::cout << process(2.16) << std::endl; // 1.08
    std::cout << sum(10, 3) << std::endl;       // 13
    std::cout << sum(10.6, 3.7) << std::endl;   // 14.3
}
```

Шаблоны с переменным числом аргументов (variadic templates)

Шаблон *variadic* — это класс или шаблон функции, поддерживающий произвольное количество аргументов

```
template <typename Head, typename... Tail>
```

В шаблонах с переменным числом аргументов многоточие используется двумя способами. Слева от имени параметра оно обозначает *пакет* параметров:

```
void print (const Head& head, const Tail&... tail)
```

, а справа от имени параметра «распаковывает» блок параметров в отдельные имена:

```
print(tail...);
```

Большинство реализаций, включающих шаблоны функций variadic, используют рекурсию. Рекурсия Variadic включает вызов шаблона функции variadic с помощью разных (почти всегда уменьшающихся) чисел аргументов и тем самым каждый раз специализируя его. "Базовый случай" по-прежнему является обязательным.

Пример

```
struct Group {
    int a = 5;
    const char* ch = "最近怎么样?";
};

std::ostream& operator << (std::ostream& out, const Group& gr) {
    return out << gr.a << ' ' << gr.ch;
}
void print () {}

template <typename Head, typename... Tail>
void print (const Head& head, const Tail&... tail) {
    std::cout << head << '\n’;
    print(tail...);
}
int main () {
    Group group;
    print ("Hello", 5, 'A’);
    print (4.6, group);
}
```

std::tuple

это шаблон класса в C++ (начиная с C++11), представляющий собой кортеж — коллекцию фиксированного размера, способную хранить значения разных типов

```
// Базовый случай: пустой кортеж
template<>
class Tuple<> {};

// Рекурсивный случай
template<typename First, typename... Rest>
class Tuple<First, Rest...> : public Tuple<Rest...> {
    First value;
public:
    Tuple(First v, Rest... rest) : value(v), Tuple<Rest...>(rest...) {}
    First& get() { return value; }
};

template<size_t N, typename First, typename... Rest>
auto& get(Tuple<First, Rest...>& t) {
    if constexpr (N == 0) {
        return t.get();  // первый элемент
    } else {
        return get<N-1>(static_cast<Tuple<Rest...>&>(t));
    }
}
```

std::tuple

```
#include <iostream>
#include <tuple>
#include <string>


int main() {
    std::tuple<int, double, std::string> t(2, 3.1, "hello");

    std::cout << std::get<0>(t) << std::endl;  // 2
    std::cout << std::get<1>(t) << std::endl;  // 3.1
    std::cout << std::get<2>(t) << std::endl;  // "hello"

    std::get<0>(t) = 100;
    std::cout << std::get<0>(t) << std::endl;  // 100

    return 0;
}
```

std::tie() — это функция, которая создаёт кортеж ссылок на переданные переменные.
Позволяет распаковать кортеж в существующие переменные или сравнивать объекты по полям.
std::make_tuple() —это функция, которая создаёт кортеж, автоматически выводя типы переданных аргументов.

```
int main() {
    int a, b;
    std::tie(a, b) = std::make_tuple(10, 20);  // a = 10, b = 20

    std::cout << a << " " << b << std::endl;  // 10 20
}
```

```
std::tuple<int, double, std::string> get_data() {
    return {12, 3.14, "hello"};
}


int main() {
    int a;
    double b;
    std::string c;

    std::tie(a, b, c) = get_data();  // a = 12, b = 3.14, c = "hello"
}
```

Если свой класс

Для корректной работы tuple в своём классе необходимо определить операторы сравнения.

```
class MyClass {
    char* str;
    int value;
public:
    //конструкторы, деструктор и оператор =

    bool operator<(const MyClass& other) const {
        return strcmp(str,other.str) < 0;
    }
    bool operator==(const MyClass& other) const {
        return strcmp(str,other.str) == 0;
    }
    friend std::ostream& operator<< (std::ostream& o, const MyClass& t) {
        return o << t.value << ' ' << t.str << '\n';
    }
};
int main () {
    auto t1 = std::make_tuple(MyClass("Alice", 1), MyClass("Mary",2));
    auto t2 = std::make_tuple(MyClass("Alice", 1), MyClass("Mary",2));
    std::cout << std::get<0>(t1) << std::endl;
    std::cout << std::get<1>(t1) << std::endl;
    if (t1 == t2) {
            std::cout << "Equal!\n";
        }
}
```